

```

"""
File Organizer
=====
Scans a source folder and moves all .jpg / .jpeg images
into a dedicated 'Images' sub-folder automatically.

Usage:
    python file_organizer.py                # organizes current directory
    python file_organizer.py /path/to/folder # organizes specified folder
"""

import os
import shutil
import sys
from datetime import datetime

def get_source_folder() -> str:
    """Return the folder to scan (CLI arg or current directory)."""
    if len(sys.argv) > 1:
        folder = sys.argv[1]
        if not os.path.isdir(folder):
            print(f" ❌ '{folder}' is not a valid directory.")
            sys.exit(1)
        return folder
    return os.getcwd()

def move_jpg_files(source_folder: str) -> None:
    """Move all .jpg / .jpeg files from source_folder into an 'Images' sub-folder."""

    dest_folder = os.path.join(source_folder, "Images")

    # Collect image files (case-insensitive)
    image_extensions = {".jpg", ".jpeg"}
    image_files = [
        f for f in os.listdir(source_folder)
        if os.path.isfile(os.path.join(source_folder, f))
        and os.path.splitext(f)[1].lower() in image_extensions
    ]

    if not image_files:
        print(" ⚠️ No .jpg / .jpeg files found in the source folder.")
        return

    # Create destination folder if it doesn't exist
    os.makedirs(dest_folder, exist_ok=True)
    print(f"\n 📁 Destination folder: {dest_folder}\n")

    moved = 0
    skipped = 0
    log_lines = []

    for filename in sorted(image_files):
        src_path = os.path.join(source_folder, filename)
        dest_path = os.path.join(dest_folder, filename)

```

```

# Avoid overwriting- rename with timestamp if conflict
if os.path.exists(dest_path):
    base, ext = os.path.splitext(filename)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    new_filename = f"{base}_{timestamp}{ext}"
    dest_path = os.path.join(dest_folder, new_filename)
    print(f" ⚠ Conflict - renaming to '{new_filename}'")

shutil.move(src_path, dest_path)
final_name = os.path.basename(dest_path)
print(f" ✅ Moved: {filename} → Images/{final_name}")
log_lines.append(f"MOVED | {filename} -> Images/{final_name}")
moved += 1

# Write a simple log file
log_path = os.path.join(source_folder, "organizer_log.txt")
with open(log_path, "a", encoding="utf-8") as log_file:
    log_file.write(f"\n--- Run: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')} ---\n")
    log_file.write("\n".join(log_lines) + "\n")

print(f"\n 📊 Summary: {moved} file(s) moved, {skipped} skipped.")
print(f" 📝 Log saved to: organizer_log.txt\n")

def main() -> None:
    print("\n" + "=" * 45)
    print(" 📁 FILE ORGANIZER 📁 ")
    print("=" * 45)

    source_folder = get_source_folder()
    print(f"\n Scanning: {source_folder}")

    move_jpg_files(source_folder)

if __name__ == "__main__":
    main()

```